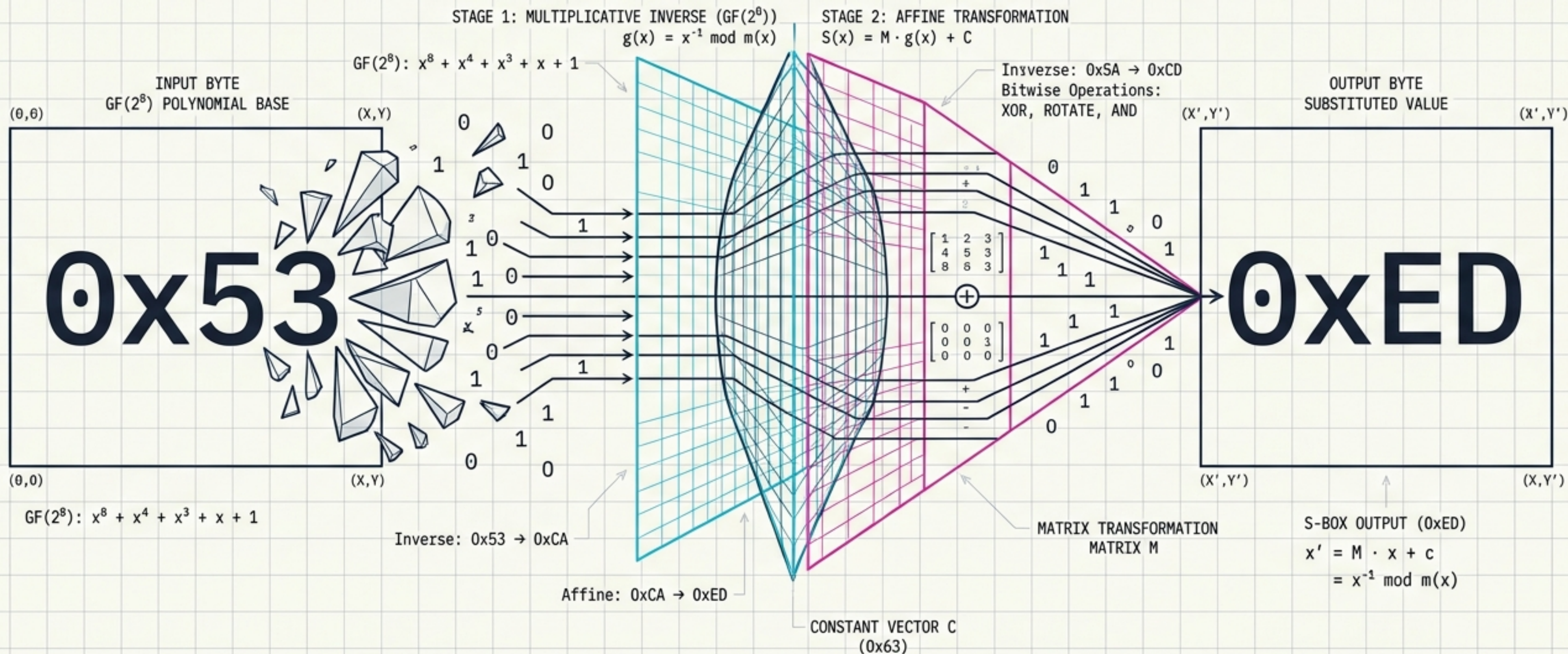


The AES S-Box: Inside the Mathematics

Demystifying the finite-field inverse and affine transformation behind AES substitution.



THE LOOKUP TABLE VS. THE MATHEMATICAL ENGINE

IMPLEMENTATION (THE MYTH)

```
static const unsigned char sbox[256] = {  
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67,  
    0x2b, 0xfe, 0xd7, 0xab, 0x76, 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59,  
    0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,  
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5,  
    0xf1, 0x71, 0xd8, 0x31, 0x15, 0x04, 0xc7, 0x23, 0x18, 0x96, 0x05,  
    0x9a, 0x07, 0x12, 0xeb, 0xeb, 0xb2, 0x75, 0x09, 0x83, 0x2c, 0x1a,  
    0x1b, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3,  
    0xe3, 0x2f, 0x84, 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0x5b, 0x6a,  
    0xcb, 0xbe, 0x39, 0x4c, 0x58, 0xcf, 0xd0, 0xef, 0xaa, 0xfb, 0x4d,  
    0x85, 0x02, 0x7f, 0x50, 0x9f, 0xa8, 0x51, 0xa3, 0x8f, 0x92, 0x9d,  
    0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2, 0xcd,  
    0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d,  
    0x4f, 0xdc, 0x2a, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b,  
    0x0b, 0x00, 0x00, 0x3a, 0x0a, 0x39, 0x38, 0x34, 0x22, 0x5c, 0xc2,  
    0x62, 0x91, 0x95, 0xe9, 0xe7, 0x37, 0x6d, 0x8d, 0x05, 0x5c, 0xf4,  
    0x7a, 0x65, 0x7a, 0xae, 0x08, 0xba, 0xba, 0x78, 0x25, 0x2e, 0x1c,  
    0xa6, 0x1f, 0x4b, 0xbd, 0x8d, 0x8b, 0x8a, 0x70, 0x3e, 0x61, 0x35,  
    0x57, 0x89, 0xb9, 0x88, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b,  
    0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf, 0xa1, 0x89, 0x0d, 0xbf,  
    0x26, 0x0f, 0xb0, 0x54, 0xbb, 0x16 };
```

A stored array of 256 substitutions. Used for speed.

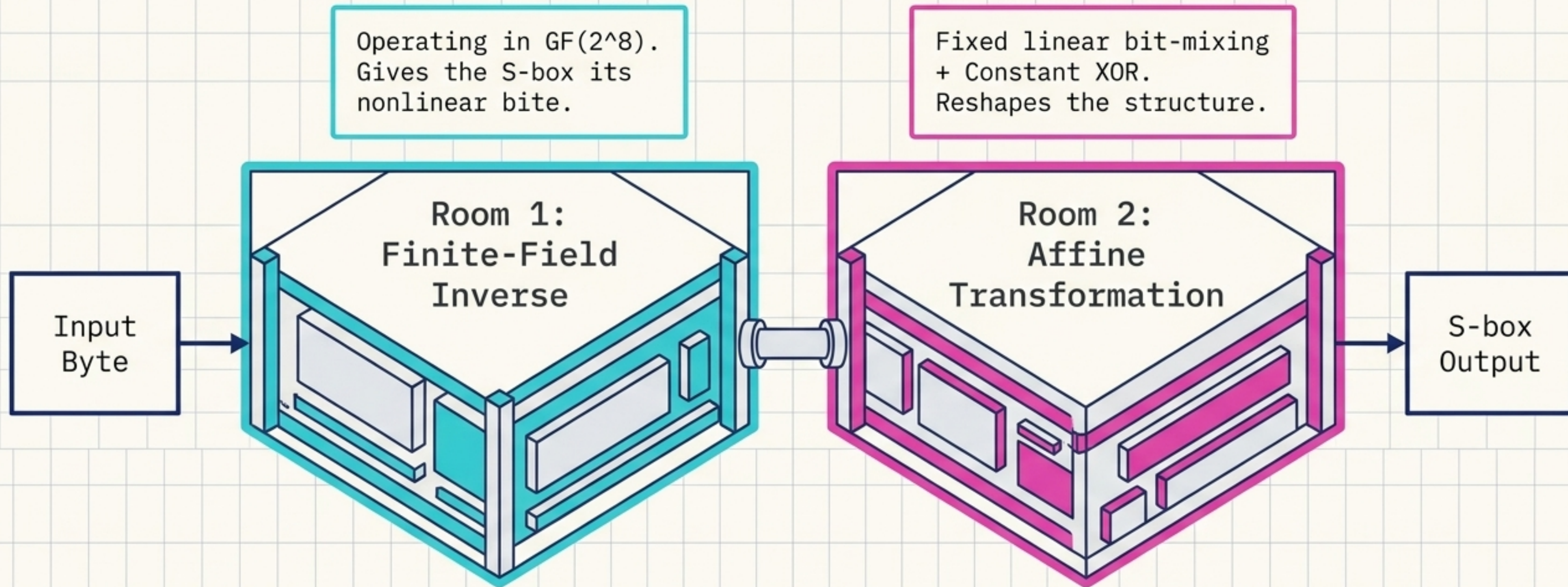
DEFINITION (THE REALITY)



A precise, two-stage mathematical construction.

A lookup table can cache the answers, but the AES S-box is fundamentally defined by structured mathematics.

THE TWO-ROOM WORKSHOP

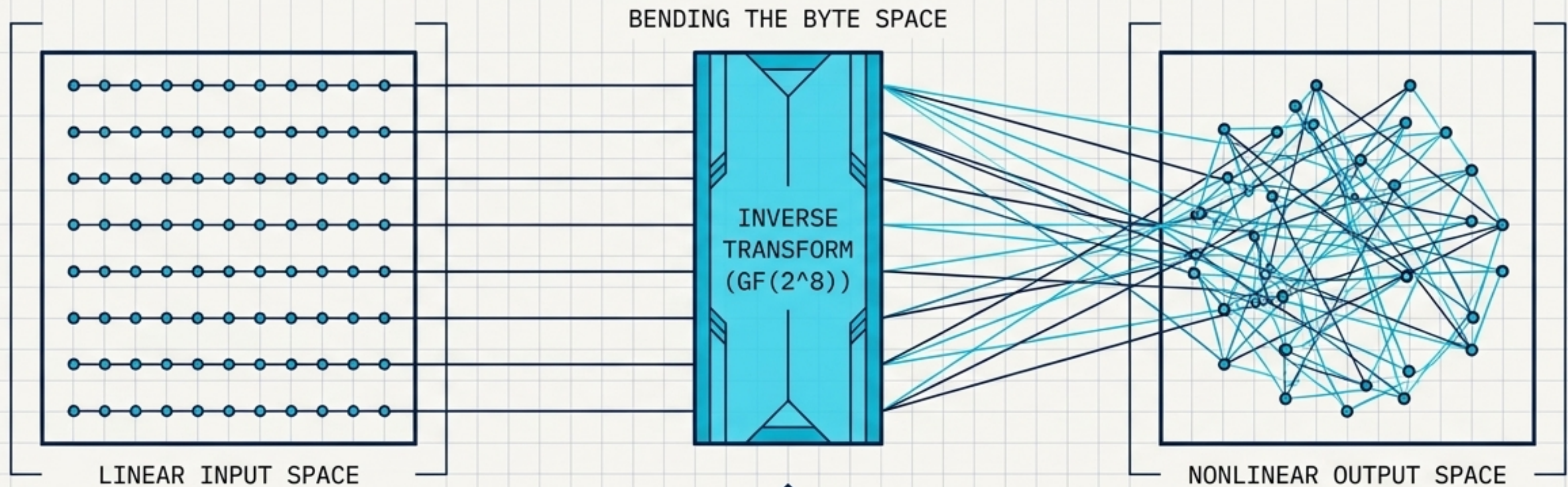


THE LEXICON

a		The original input byte in GF(2 ⁸)
a⁻¹		The finite-field inverse of a
b_i		Bit i of the inverse byte
c_i		Bit i of the constant byte 0x63
s_i		Bit i of the S-box output
⊕		XOR (Addition modulo 2)

Room 1: The Finite-Field Inverse

$$a \cdot a^{-1} = 0x01$$



For any nonzero input byte a , AES computes its multiplicative inverse in $GF(2^8)$. This mathematical operation aggressively bends the data, destroying linear relationships.

THE BOUNDARY EXCEPTION

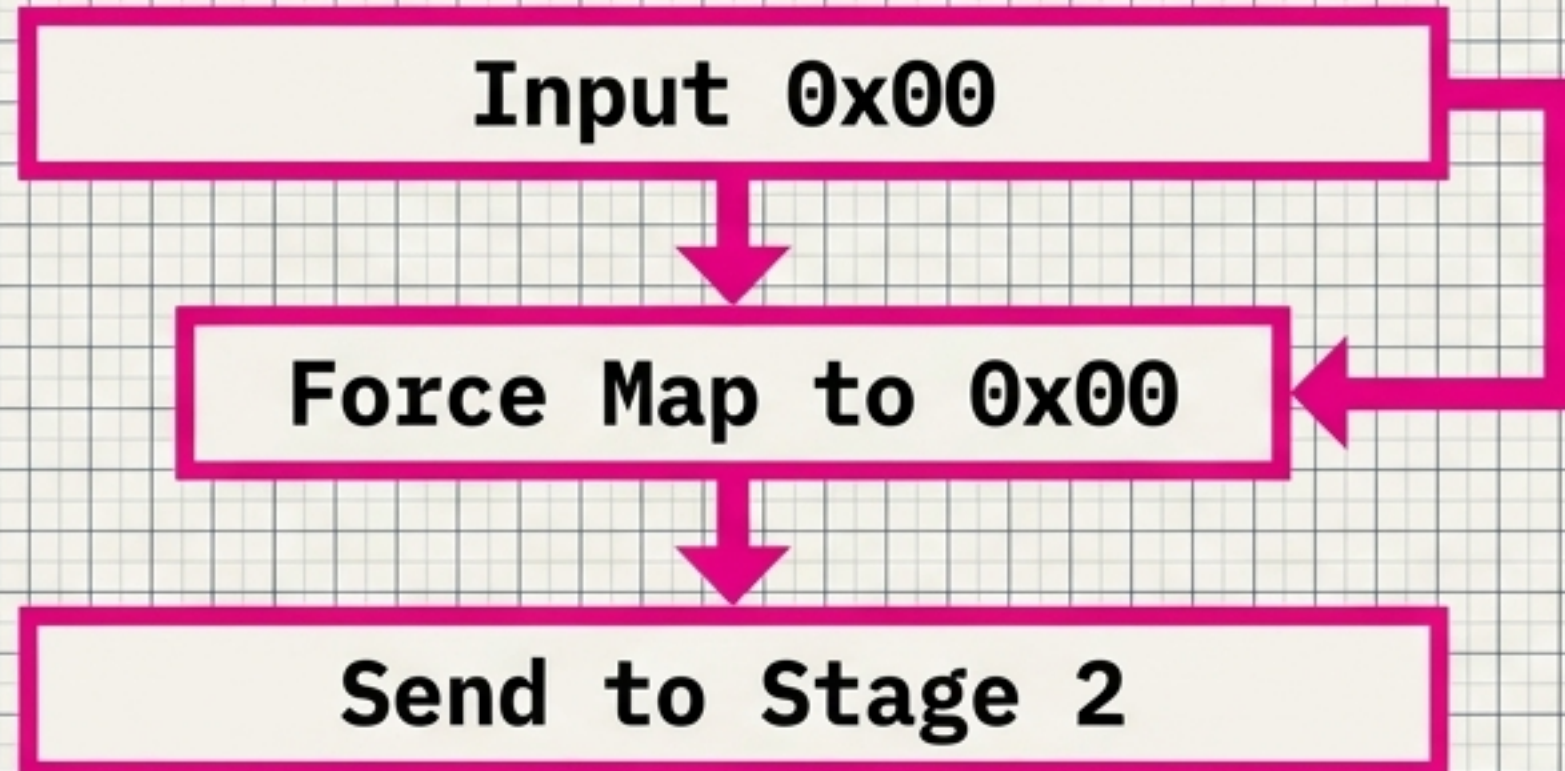
The Problem

$$\cancel{0x00 \cdot ? = 0x01}$$



The zero byte has no multiplicative inverse.

The AES Rule



AES does not claim 0x00 has an inverse. It explicitly defines the inverse step for 0x00 as 0x00, ensuring the S-box successfully maps every possible byte.

Room 2: The Affine Transformation

Affine Transformation

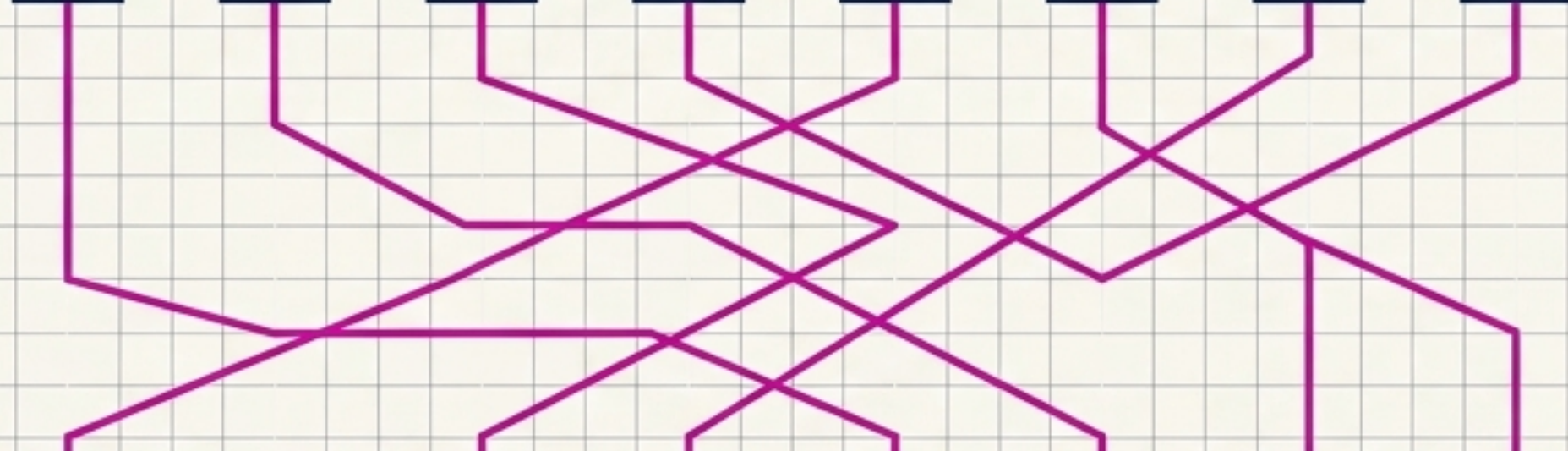
=

Linear Bit-Mixing

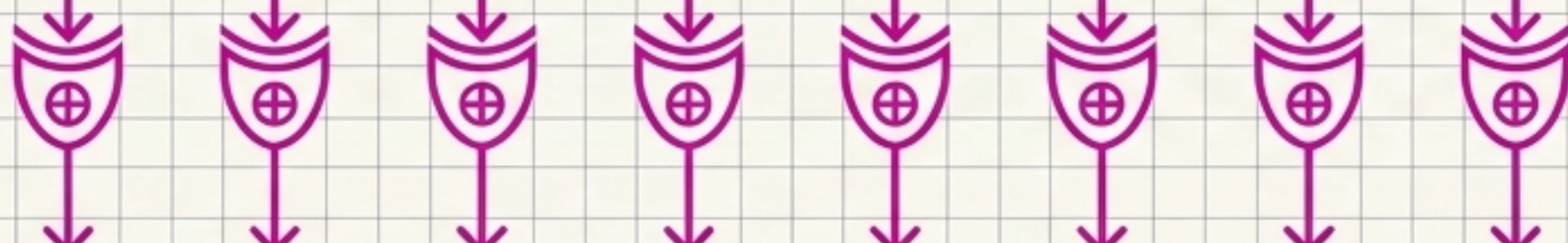
+

Constant XOR

Input Byte



Constant
0x63



Output Byte



Deconstructing the Constant

0x63

Bit 7 6 5 4 3 2 1 Bit 0



Standard MSB-to-LSB display

Bit 0 1 2 3 4 5 6 Bit 7



Convenient LSB-to-MSB index for the math formula

Bit 0 represents the 1s place (Least Significant Bit). The affine formula heavily relies on strict indexing—mixing these directions up is a guaranteed failure.

Two Views of the Affine Step

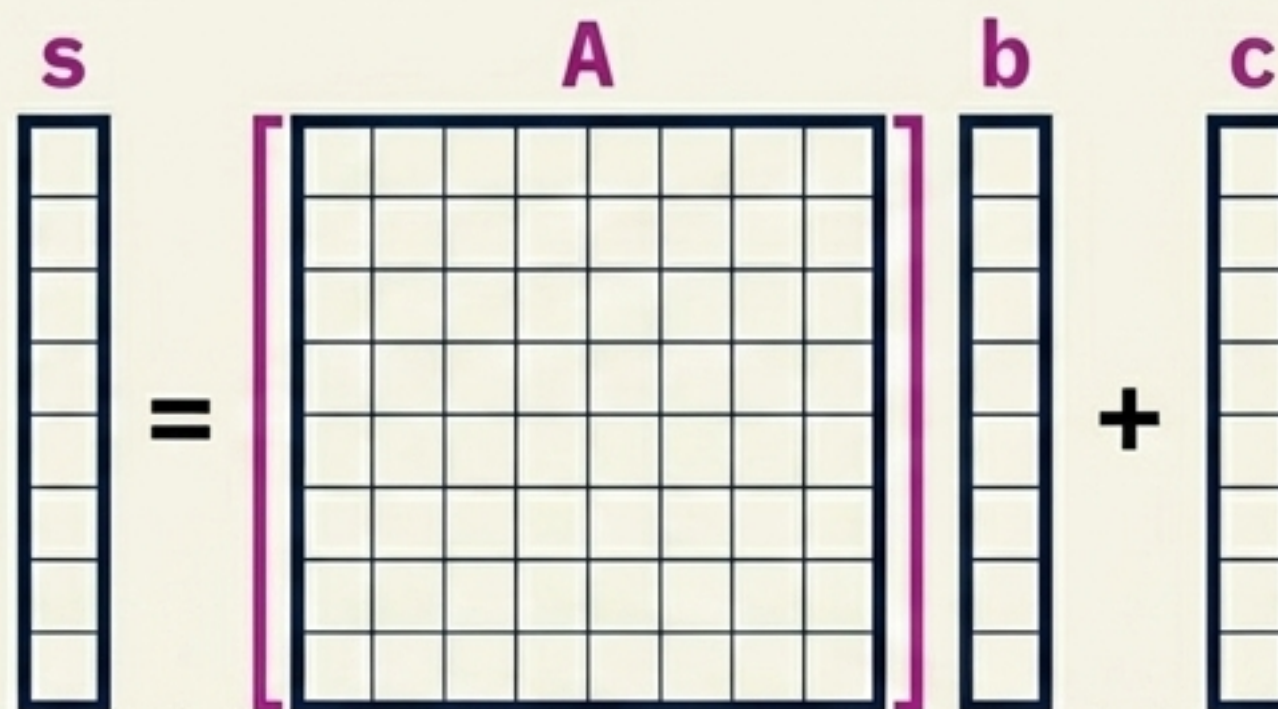
The Bit Formula

$$s_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \\ \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i$$

Best for hand calculation and software logic.

The Matrix View

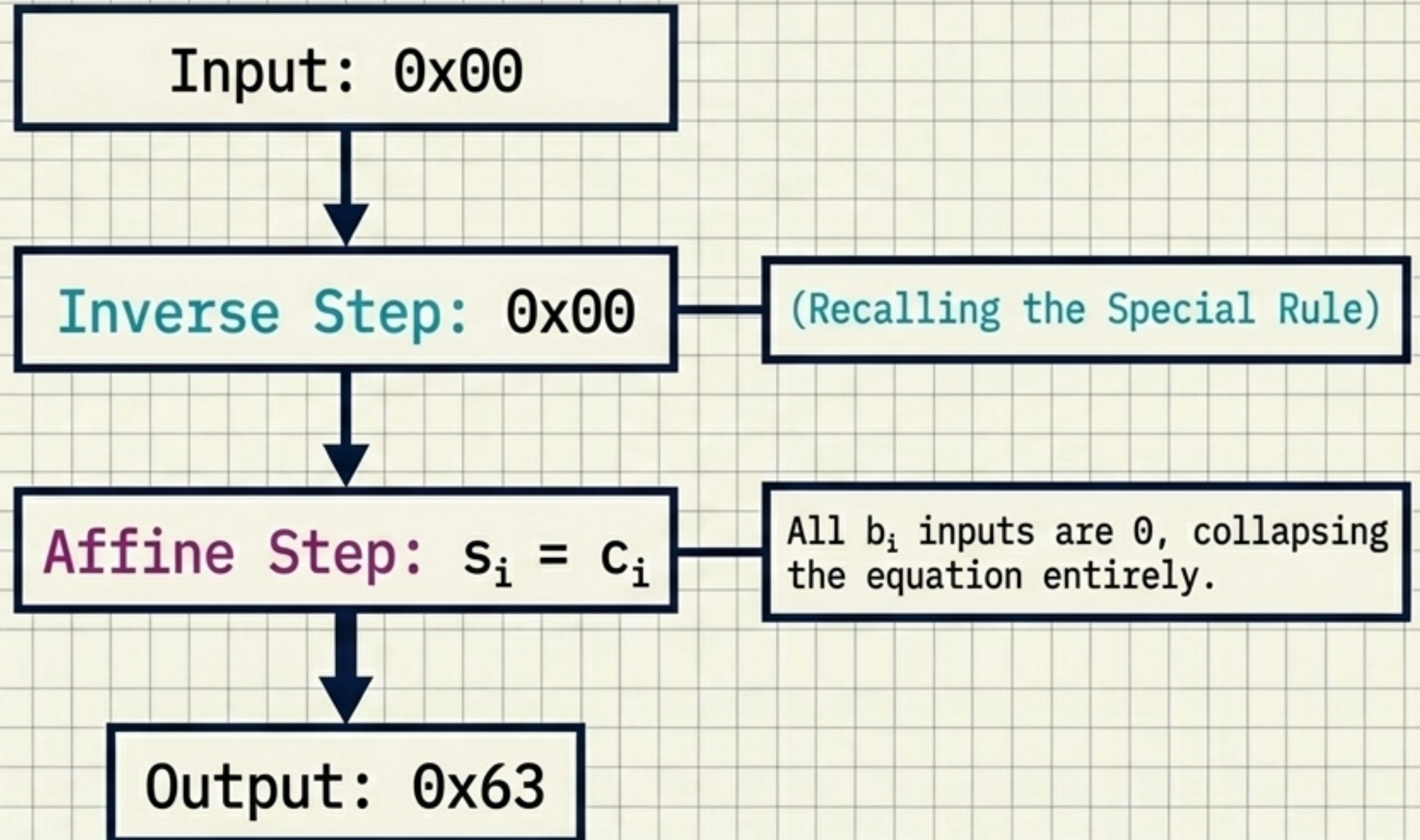
$$s = Ab + c$$



Proves it is a structured linear transformation.

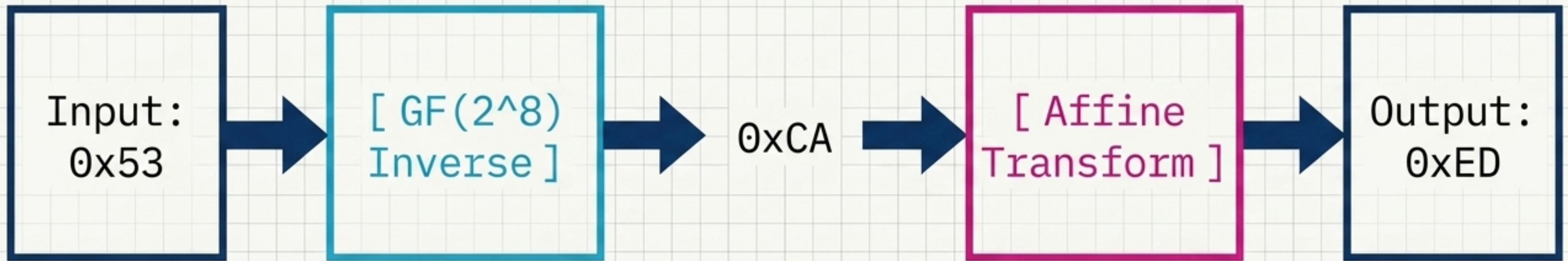
The bit formula is easier to compute by code. The matrix form proves the step is a highly structured linear transformation plus a constant vector over $\text{GF}(2)$.

Case A: The Zero Byte



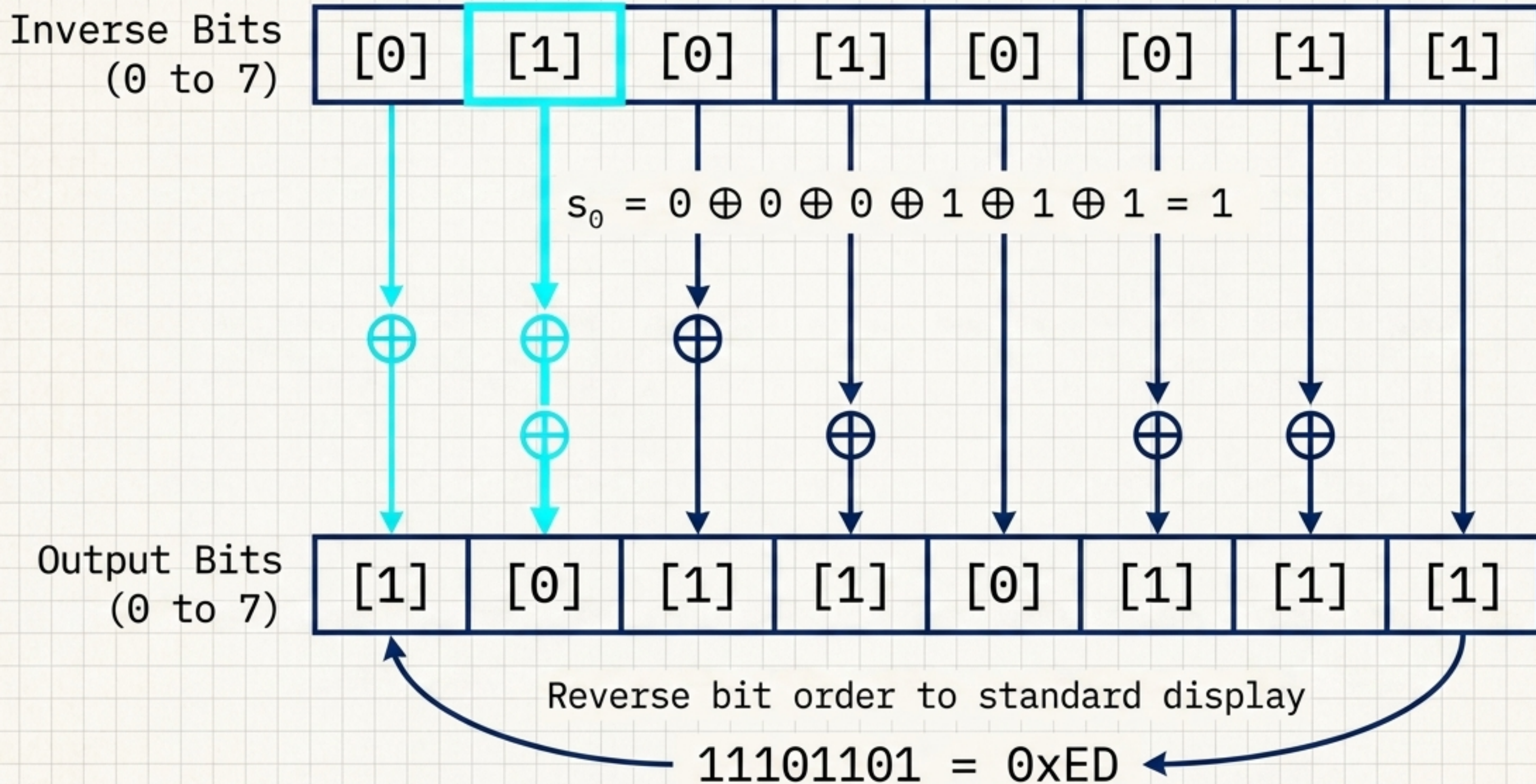
Because all inverse bits are zero, the entire linear transformation zeroes out, leaving only the affine constant.

Case B: The Standard Test Case



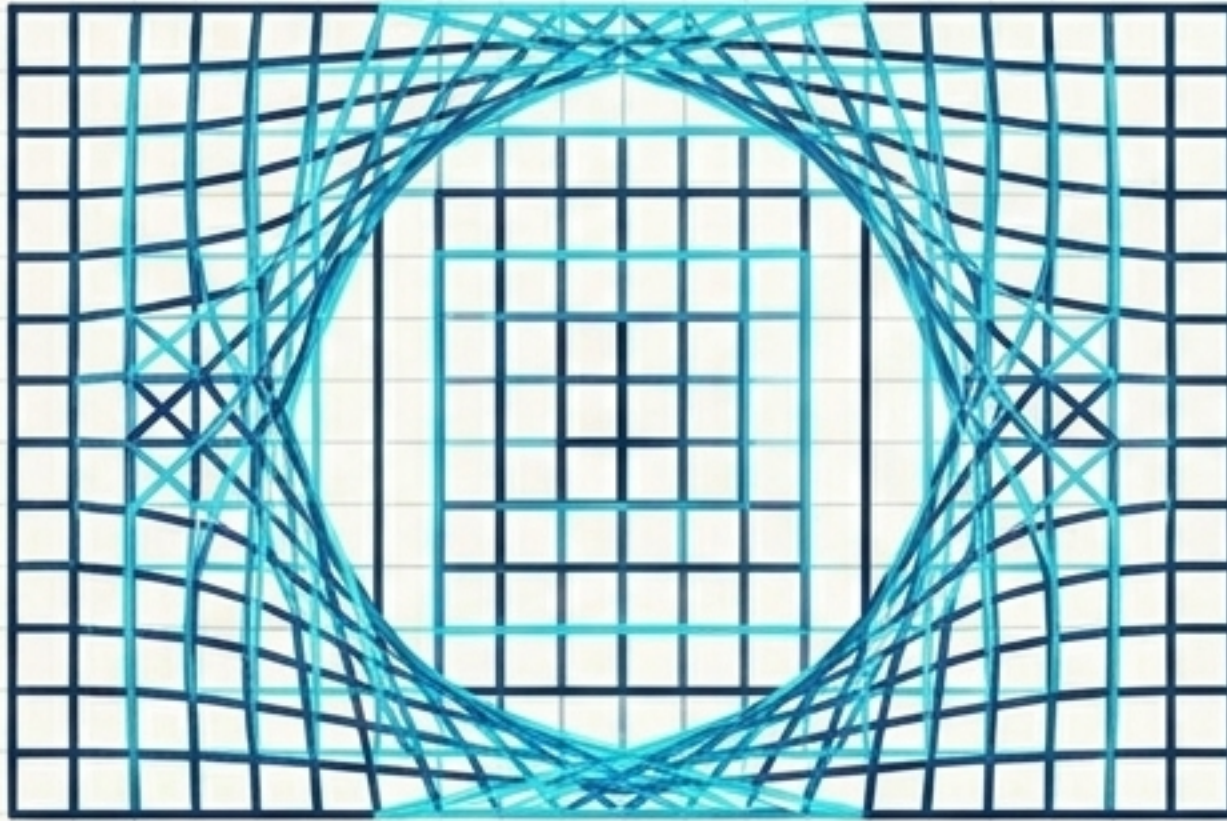
The finite-field inverse of `0x53` is `0xCA`. The affine step maps `0xCA` to `0xED`.
(Next, we look directly inside the affine math for `0xCA`).

Inside the Bit-Mixing Matrix



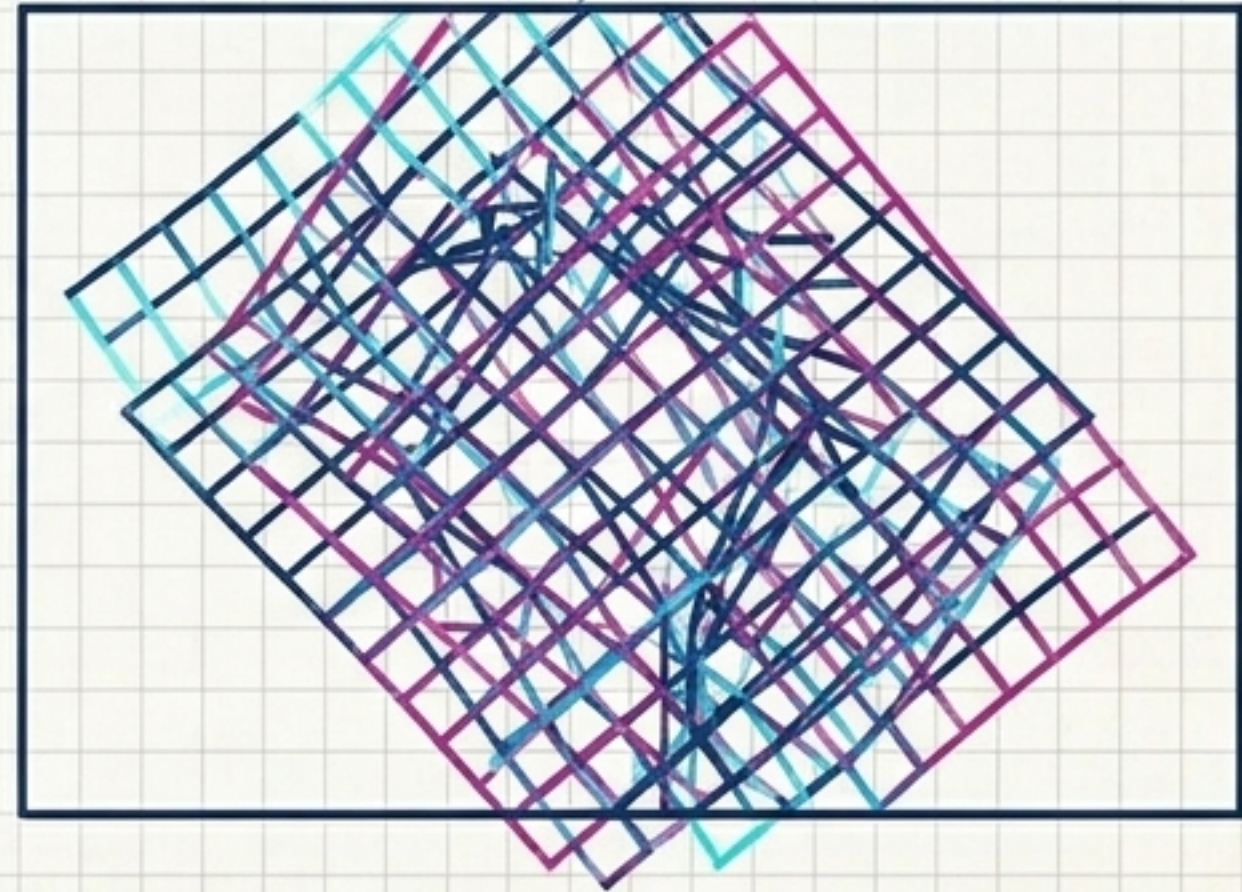
The Purpose of the Affine Step

Inverse Alone



Creates nonlinearity, but leaves awkward algebraic symmetries.

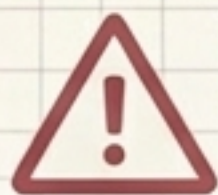
Inverse + Affine



Rotates and offsets the bent space to eliminate fixed patterns.

The inverse step bends the byte space for nonlinearity. The affine step rotates and offsets that bent space, removing undesirable algebraic symmetries and improving cryptographic behavior. It is purposeful finishing work.

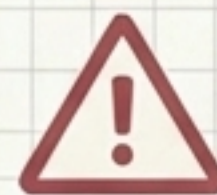
Diagnostic: 4 Common Pitfalls



Mistake: Stopping at the inverse.



Correction: The **inverse** is **only Stage 1**. You must apply the **affine transformation**.



Mistake: Trying to invert $0x00$.



Correction: $0x00$ has no inverse. Map it to $0x00$ before the affine step.



Mistake: Mixing up bit order.



Correction: The formula indexes **bit** 0 as the least significant bit. Convert carefully.



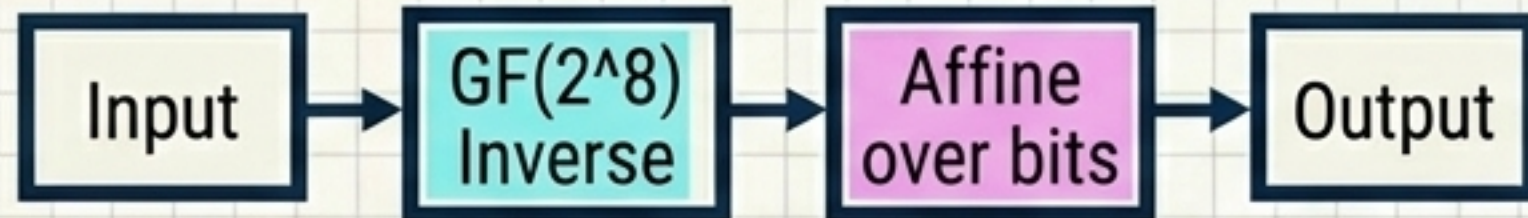
Mistake: Using standard arithmetic.



Correction: **Affine** uses XOR over bits, never standard decimal addition.

The A-Box S-Box Cheat Sheet

The Pipeline



The Master Formula

$$s_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \\ \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i$$
$$c = 0x63 = 01100011$$

Case Matrix

Input	Inverse	Affine	S-box Output
[0x00]	[0x00]	[0x63]	[0x63]
[0x53]	[0xCA]	[0xED]	[0xED]

AES S-Box: Nonlinear substitution driven by finite fields and structured bit geometry.